```
def labeled_form_for(record, options = {}, &block)
  options.merge! builder: LabellingFormBuilder
  form_for record, options, &block
end
```

The form builder used also determines what happens when you do

```
<%= render partial: f %>
```

If `f` is an instance of `FormBuilder` then this will render the `form` partial, setting the partial's object to the form builder. If the form builder is of class `LabellingFormBuilder` then the `labelling_form`partial would be rendered instead.

7 Understanding Parameter Naming Conventions

As you've seen in the previous sections, values from forms can be at the top level of the `params` hash or nested in another hash. For example, in a standard `create` action for a Person model, `params[:person]` would usually be a hash of all the attributes for the person to create.
The `params`hash can also contain arrays, arrays of hashes and so on.
Fundamentally HTML forms don't know about any sort of structured data, all they generate is name-value pairs, where pairs are just plain strings. The arrays and hashes you see in your application are the result of some parameter naming conventions that Rails uses.

You may find you can try out examples in this section faster by using the console to directly invoke Rack's parameter parser. For example,

```
Rack::Utils.parse_query "name=fred&phone=0123456789"
# => {"name"=>"fred", "phone"=>"0123456789"}
```

7.1 Basic Structures

The two basic structures are arrays and hashes. Hashes mirror the syntax used for accessing the value in `params`. For example, if a form contains:

```
<input id="person_name" name="person[name]" type="text" value="Henry"/>
```

the `params` hash will contain

```
{'person' => {'name' => 'Henry'}}
```

and `params[:person][:name]` will retrieve the submitted value in the controller.
Hashes can be nested as many levels as required, for example:

```
<input id="person_address_city" name="person[address][city]" type="text"
value="New York"/>
```

will result in the `params` hash being

```
{'person' => {'address' => {'city' => 'New York'}}}
```

Normally Rails ignores duplicate parameter names. If the parameter name contains an empty set of square brackets `[]` then they will be accumulated in an array. If you wanted users to be able to input multiple phone numbers, you could place this in the form:

```
<input name="person[phone_number][]" type="text"/>
<input name="person[phone_number][]" type="text"/>
<input name="person[phone_number][]" type="text"/>
```

This would result in `params[:person][:phone_number]` being an array containing the inputted phone numbers.